Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Cryptanalysis

Christophe Petit

Université libre de Bruxelles

Sensitization campaign

Together against sexual harassment and violences
https://www.ulb.be/fr/sante-et-bien-etre/si-cest-pas-oui-cest-non

Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Cryptanalysis

Christophe Petit

Université libre de Bruxelles

Cryptography vs Cryptanalysis

- Cryptography is the science and art of ensuring private and authenticated communications
- Cryptanalysis is the science and art of breaking it!

- Cryptanalysis is useful for cryptography
  - Cryptanalysis attempts give confidence on hardness
  - Cryptanalysis benchmarks help determining key sizes
  - Cryptanalysis successes teach us design lessons

  Good criminals make good policemen too !

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Remember: "Provable Security"

- Precisely define what it means to break the protocol
  - Adversary's goal
  - Adversary's resources
  - Adversary's access to the system
- Choose your favorite hard problem
  - A computational problem that cannot be solved,
    even by clever people with the best computers available
- Build a protocol so that you can prove

  Breaking the protocol $\Rightarrow$ Solving the hard problem

Limits in Provable Security

- Remember "provable security"

  Breaking the protocol $\Rightarrow$ Solving the hard problem

- How can this go wrong?
  - Find out that security proof is wrong
  - Solve the underlying hard problem
  - Find weaknesses not covered by the proof

Why solving hard problems?

- Remember "provable security"

  Breaking the protocol $\Rightarrow$ Solving the hard problem

- Proof only useful if the problem is indeed "hard"
- Can often improve security at the price of efficiency
- Need cryptanalysis benchmark to select the key sizes optimizing security and efficiency

How to solve hard problems?

- Optimize existing algorithms and design new ones
- Implement algorithms on all sorts of computing devices
- Benchmark cryptanalysis results
- Try and engage more people to try!

A glance at a cryptanalyst's toolbox

- Computational number theory
- Birthday paradox attacks
- Linear and differential cryptanalysis
- Generic discrete logarithm algorithms
- Index calculus algorithms
- Lattice reduction algorithms
- Groebner basis algorithms, SAT solvers,...
- Anything that can help!

A glance at a cryptanalyst's toolbox (2)

- Laptops and desktops
- Computing servers
- FPGAs
- Dedicated hardware
- Sony Playstations
- Cloud computing
- Mobile phones
- Anything that can help!

Playstation games for cryptanalysts

- Advanced attacks on MD5 led to Nostradamus attack

Public challenges

- Goal: encourage cryptanalysis attempts to estimate concrete hardness of a problem, and parameters to be used in practice
  - RSA factoring challenges (ended 2007)
  - Certicom ECDLP challenges (solved up to 109 bits)
  - Darmstadt lattice challenges
  - Fukuoka MQ challenges
- Contribute and enter "Hall of Fame" of any of them!

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Birthday paradox

- Suppose there are N_2 people in a room. What is the probability that two people have the same birthday?
- How many people needed to have a probability larger than 50% ?
- Answer is **23**:

$$\Pr[\text{all distinct}] = 1 \cdot \frac{364}{365} \cdot \frac{363}{365} \cdot \ldots \cdot \frac{365-22}{365} < \frac{1}{2}$$

Birthday paradox

- Suppose you choose N_2 elements randomly in a set of N elements. What is the probability that two elements are equal?
- How should N_2 be wrt N to have a probability larger than 50% ?
- Answer is $O(\sqrt{N})$:

$$\begin{aligned}
\Pr[\text{all distinct}] &= 1 \cdot \frac{N-1}{N} \cdot \frac{N-2}{N} \cdot \ldots \cdot \frac{N-N_2+1}{N} \\
&\approx e^{-\frac{1}{N}} \cdot e^{-\frac{2}{N}} \cdot \ldots \cdot e^{-\frac{N_2-1}{N}} \\
&\approx e^{-\frac{N_2(N_2-1)}{N}}
\end{aligned}$$

Taking $N_2 \approx \sqrt{N}$ ensures $1 - \Pr[\text{all distinct}] \in \Omega(1)$

Computing collisions

- Remember: a hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is collision resistant if it is hard to find (m, m') with $H(m) = H(m')$ but $m \neq m'$
- Simple algorithm: choose random messages m_i and compute $h_i = H(m_i)$ until you get a collision
- By birthday paradox, you only need $O(2^{n/2})$ elements to get a collision

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Discrete logarithm algorithms

- Remember discrete logarithm problem (DLP): given a finite cyclic group G of prime order p, a generator g and a second element h, find k such that $h = g^k$
- Hardness depends on the group
- Pollard's rho, Baby-Step-Giant-Step algorithms work for any group, with complexity $O(\sqrt{|G|})$
- These are current best for well-chosen elliptic curves
- Index calculus algorithms have complexity $\leq L_{|G|}(1/3, c)$ when G is multiplicative group of finite field
- Much better complexity when characteristic is small (quasi-polynomial time algorithm)

Exhaustive search

- Let $h = g^k$. You want to compute k.
- Exhaustive search algorithm

```
1: k ← 1; h' ← g
2: if h' = h then
3:     return k
4: else
5:     k ← k + 1; h' ← h'g
6:     Go to Step 2
7: end if
```

- Generic algorithm
- Time complexity $|G|$ in the worst case, $|G|/2$ on average
- Can we do better?

Baby step, giant step (BSGS)

- Let $h = g^k$. You want to compute k.
- Let $N' = \lceil \sqrt{|G|} \rceil$
- There exist $0 \leq i, j < N'$ such that $k = jN' + i$

$$h = g^{jN'+i} \Leftrightarrow hg^{-jN'} = g^i$$

- Compute $L_B := \{g^i | i = 0, \ldots, N' - 1\}$
- Compute $L_G := \{hg^{-jN'} | j = 0, \ldots, N' - 1\}$
- Find an element in $L_G \cap L_B$
- Attack requires time and memory $O(\sqrt{|G|})$

Pollard's rho (iterative function)

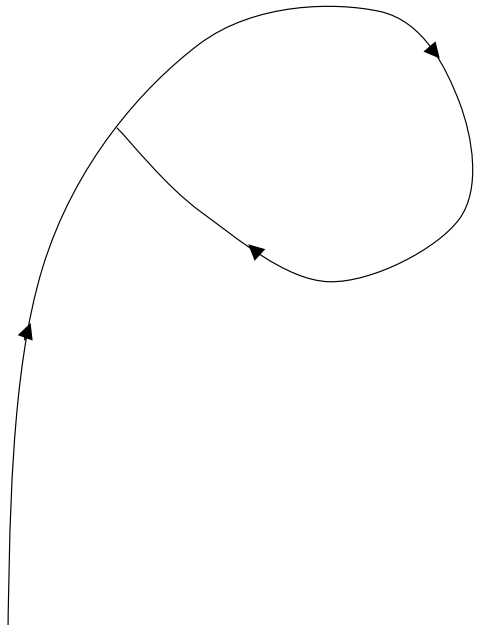
- Define G_1, G_2, G_3 of about the same size such that $G = G_1 \cup G_2 \cup G_3$ and $G_i \cap G_j = \{\}$
- Over $\mathbb{Z}_p^*$, can choose
  $G_1 = \{0, \ldots, \lfloor p/3 \rfloor\}$,
  $G_2 = \{\lfloor p/3 \rfloor + 1, \ldots, \lfloor 2p/3 \rfloor\}$,
  $G_3 = \{\lfloor 2p/3 \rfloor + 1, \ldots, p-2\}$
- Define a function $f : G \to G$ such that

$$\begin{cases} f(z) = zg & z \in G_1 \\ f(z) = z^2 & z \in G_2 \\ f(z) = zh & z \in G_3 \end{cases}$$

(original definition, but other definitions possible)

Pollard's rho (intuition)

- Start from $g_0 := g$ and apply f recursively to get g_i
- If f is "random enough", we obtain random elements in G and a collision after $O(\sqrt{|G|})$ elements
- By the way f is defined, we can keep track of a_i, b_i such that $g_i = g^{a_i} h^{b_i}$
- Collision gives DLP solution



Pollard's rho (simplest version)

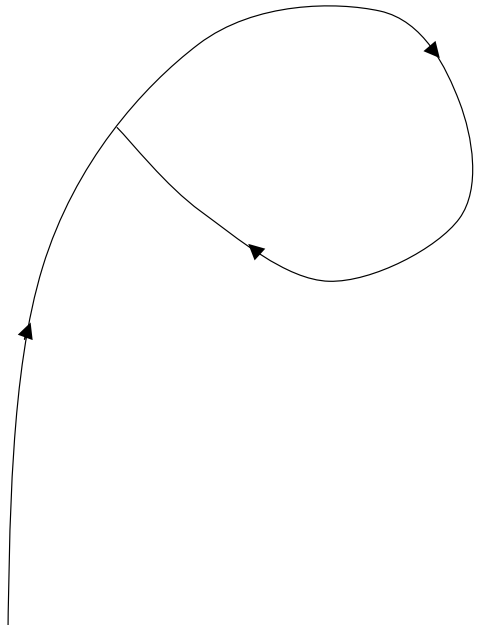
$N \leftarrow \lceil \sqrt{|G|} \rceil$
$a \leftarrow 1; b \leftarrow 0; \tilde{h} \leftarrow g; L \leftarrow \{(a, b, \tilde{h})\}$
for $k \in \{2, \ldots, N\}$ **do**
    if $\tilde{h} \in G_1$ **then** $a \leftarrow a + 1; \tilde{h} \leftarrow \tilde{h}g$
    if $\tilde{h} \in G_2$ **then** $a \leftarrow 2a; b \leftarrow 2b; \tilde{h} \leftarrow (\tilde{h})^2$
    if $\tilde{h} \in G_3$ **then** $b \leftarrow b + 1; \tilde{h} \leftarrow \tilde{h}h$
    $L \leftarrow L \cup \{(a, b, \tilde{h})\}$
end for
Find distinct $(a_i, b_i, \tilde{h}) \in L$, $i = 1, 2$
if no such elements **then abort**
return $-(a_1 - a_2)/(b_1 - b_2) \bmod |G|$

Pollard's rho analysis

- Correctness:
  - Every $(a, b, \tilde{h})$ in the list satisfies $\tilde{h} = g^a h^b$
  - $g^{a_1} h^{b_1} = g^{a_2} h^{b_2}$ implies $h = g^{-\frac{a_1 - a_2}{b_1 - b_2}}$
- Time and memory costs $O(N) = O(\sqrt{|G|})$
- Good probability of success by birthday's paradox

Pollard's rho (improvement)

- Let $(L_1, L_1 + L_2)$ be the indices of first collision
- Then $(L_1 + j, L_1 + j + kL_2)$ also collide
- For j, k such that $L_1 + j = kL_2$, we have $L_1 + kL_2 + j = 2(L_1 + j)$
- Now search for $(a_i, b_i, \tilde{h}_i)$ and $(a_{2i}, b_{2i}, \tilde{h}_{2i})$ such that $\tilde{h}_i = \tilde{h}_{2i}$
- Only requires constant size memory



Pohlig-Hellman

- Assume $|G| = n_1 n_2$ and let g a generator of G
- $h = g^k$ implies $h^{n_1} = (g^{n_1})^k$
  where g^{n_1} generates a subgroup of order n_2
- Solving DLP in that subgroup gives k mod n_2
- Repeat for each (power of prime) factor of $|G|$
- Use a constructive version of Chinese Remainder Theorem (extended Euclide algorithm) to get k (see next slides)
- Countermeasure: use a large prime order group

Pohlig-Hellman: example

- Let $p := 11$
- Then 2 is a generator of $\mathbb{F}_p^*$
  - Successive powers are 2,4,8,5,10,9,7,3,6,1
  - Order is $p - 1 = 2 \cdot 5$
- Let $g = 2$ and $h = 7$
  - $g^5 = 10$, $h^5 = 10$ hence $x = 1 \bmod 2$
  - $g^2 = 4$, $h^2 = 5$ hence $x = 2 \bmod 5$
  - Deduce $x = 7$
- Let $g = 2$ and $h = 3$
  - $g^5 = 10$, $h^5 = 1$ hence $x = 0 \bmod 2$
  - $g^2 = 4$, $h^2 = 9$ hence $x = 3 \bmod 5$
  - Deduce $x = 8$

Chinese Remainder Theorem

- Let p_1, p_2 prime and let $n = p_1 p_2$. For any x_1, x_2, there is one and only one value of x modulo n such that $x = x_1 \bmod p_1$ and $x = x_2 \bmod p_2$
- This value can be efficiently computed
  1. Compute $(p_1^{-1} \bmod p_2)$ and $(p_2^{-1} \bmod p_1)$ using the Extended Euclide algorithm (see next slide)

     $$1 = p_1(p_1^{-1} \bmod p_2) + p_2(p_2^{-1} \bmod p_1)$$

  2. Deduce

     $$x = x_2 p_1(p_1^{-1} \bmod p_2) + x_1 p_2(p_2^{-1} \bmod p_1)$$

Extended Euclide algorithm

- Goal: compute d, r, s such that $d = ra + sb = \gcd(a, b)$

  Require: $a \geq b$

  Ensure: $d = \gcd(a, b)$ **and** r, s, such that $ar + bs = d$

  if $b|a$ **then**
   return $a, 0, 1$
  else
   Compute q such that $0 < a - qb < b$
   $d, r, s \leftarrow \gcd(b, a - qb)$
   return $d, s, r - qs$
  end if

- Indeed if $rb + s(a - qb) = d$ then $sa + (r - qb)b = d$
- Complexity $O(|a|^2)$; best algorithms achieve $O(|a| \log |a|)$

Extended Euclide algorithm: example

- Consider $a = 31$, $b = 11$
- We have

$$\begin{aligned} 31 &= 2 \cdot 11 + 9 \\ 11 &= 1 \cdot 9 + 2 \\ 9 &= 4 \cdot 2 + 1 \\ 2 &= 2 \cdot 1 \end{aligned}$$

- Hence

$$\begin{aligned} 9 &= 1 \cdot 31 - 2 \cdot 11 \\ 2 &= 1 \cdot 11 - 1 \cdot 9 = -1 \cdot 31 + 3 \cdot 11 \\ 1 &= 1 \cdot 9 - 4 \cdot 2 = 5 \cdot 31 - 14 \cdot 11 \end{aligned}$$

Chinese Remainder Theorem (2)

- More generally if $n = \prod_{i=1}^{N} p_i^{e_i}$, for any $x_1, x_2, \ldots, x_N$, there is one and only one value of x modulo n such that $x = x_i \bmod p_i^{e_i}$
- This value can be efficiently computed

Generic DLP algorithms

- Exhaustive search, Pollard's rho, Baby-step-giant-step and Pohlig-Hellman algorithms work **for any group**
- For any group G, solving DLP in G costs at most $O(\sqrt{q})$ group operations, where q is largest prime dividing $|G|$
- Countermeasure: take a large prime order group

DLP algorithms for finite fields

- In cryptography we often use multiplicative groups of finite fields
- For $\mathbb{F}_p^*$ the best algorithm has complexity[1]

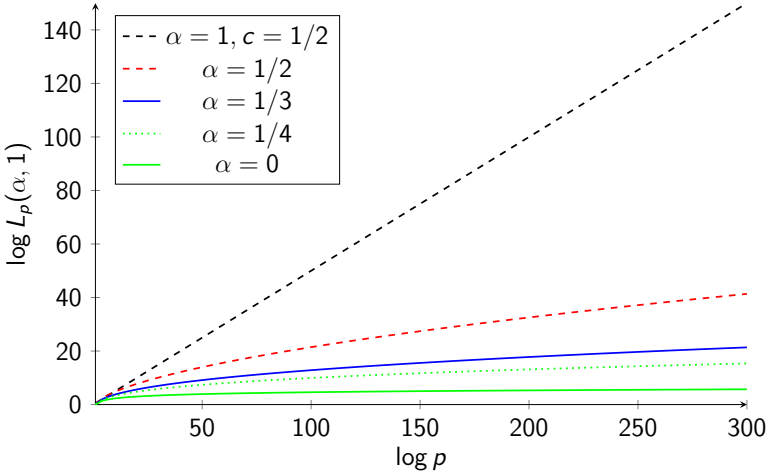
$$L_p\left(\frac{1}{3}, \sqrt[3]{\frac{64}{9}}\right) = \exp\left(\left(\sqrt[3]{\frac{64}{9}} + o(1)\right)(\log p)^{1/3}(\log\log p)^{2/3}\right)$$

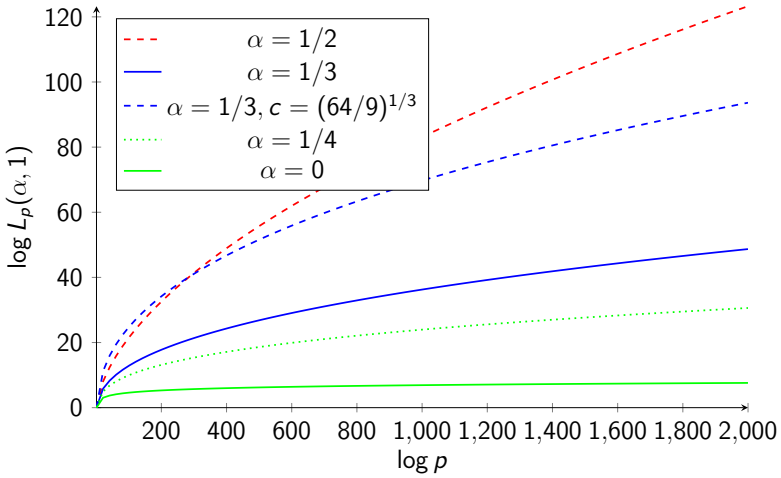
- Current record has 795 bits (Dec 2019)
- For $\mathbb{F}_{2^n}^*$ best algorithm has quasi-polynomial complexity since 2013, broken in theory and in practice

[1] $L_p(\alpha, c) = \exp((c + o(1))(\log p)^{\alpha}(\log\log p)^{1-\alpha})$

Subexponential L function

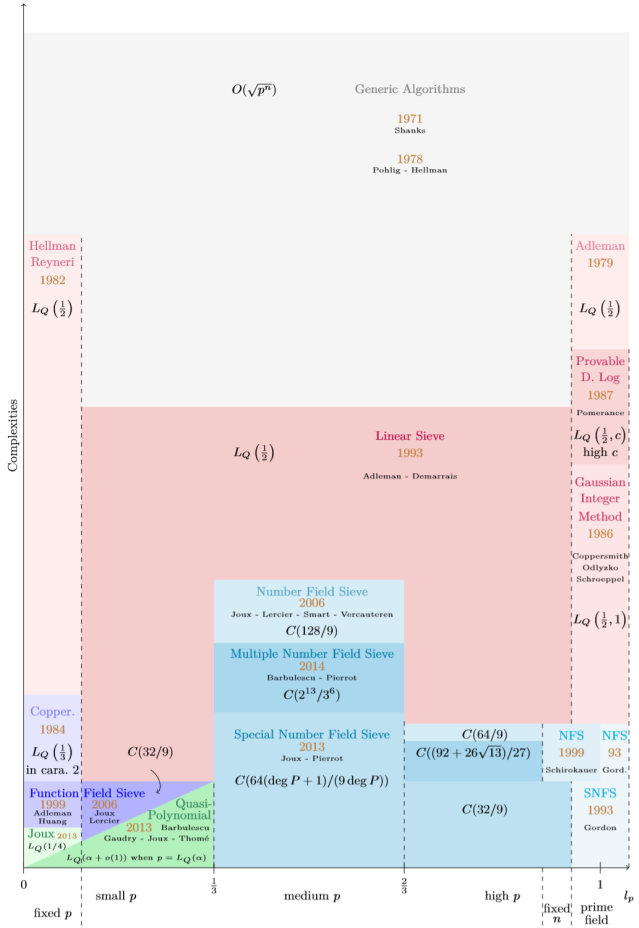
$$L_p(\alpha, c) = \exp((c + o(1))(\log p)^{\alpha}(\log\log p)^{1-\alpha})$$





Diffie-Hellman paper initially suggested 200-bit primes

DLP algorithms for finite fields



Picture credit: Joux-Odlyzko-Pierrot

Index calculus

- Generic framework to solve discrete logarithm problems, but some steps are group-specific
- Let g, h a DLP problem
- Define a *factor basis* $\mathcal{F} \subset G$, ensuring $\mathcal{F}$ contains a generator (most elements in G are generators)
- Can assume $g \in \mathcal{F}$, otherwise do the following:
  - Pick a generator $g' \in \mathcal{F}$
  - Compute a such that $g = (g')^a$
  - Compute b such that $h = (g')^b$
  - Compute $k = b/a \bmod |G|$
- Remark: size of $\mathcal{F}$ will be optimized for efficiency

Index calculus

- Find about $|\mathcal{F}|$ *relations* between factor basis elements

$$\mathcal{R}_j : \quad \prod_{f_i \in \mathcal{F}} f_i^{a_{i,j}} = 1$$

(the algorithm to compute the relations is group-specific)

- Deduce

$$\sum_{f_i \in \mathcal{F}} a_{i,j} \log_g f_i = 0$$

or

$$\begin{pmatrix} a_{1,1} & \dots & a_{|\mathcal{F}|,1} \\ \vdots & & \vdots \\ a_{1,|\mathcal{F}|} & \dots & a_{|\mathcal{F}|,|\mathcal{F}|} \end{pmatrix} \begin{pmatrix} \log_g f_1 \\ \vdots \\ \log_g f_{|\mathcal{F}|} \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

Index calculus

- Use linear algebra to compute all $\log_g f_i$,
  the discrete logarithms of factor basis elements
- Deduce the discrete logarithm of h
  (this part is group-specific and may involve several steps)
- Remarks:
  - Relations often involve few elements,
    hence linear algebra is sparse
  - In some cases, h is included in the factor basis and the last step is avoided: linear algebra produces $\log_g h$

Example: a naive index calculus for $\mathbb{F}_p^*$

- DLP: given $g, h \in \mathbb{F}_p^*$, find k such that $h = g^k$
- Factor basis made of **small primes** and h

$$\mathcal{F}_B := \{\text{primes } p_i \leq B\} \cup \{h\}$$

- **Relation search**
  - Compute $r_j := g^{a_j} h^{b_j}$ for random $a_j, b_j \in \{1, \ldots, p-1\}$
  - **If** all factors of r_j are $\leq B$, we have one relation

$$g^{a_j} h^{b_j} = \prod_{p_i \in \mathcal{F}} p_i^{e_{i,j}}$$

- **Linear algebra** produces a, b such that $g^a h^b = 1$

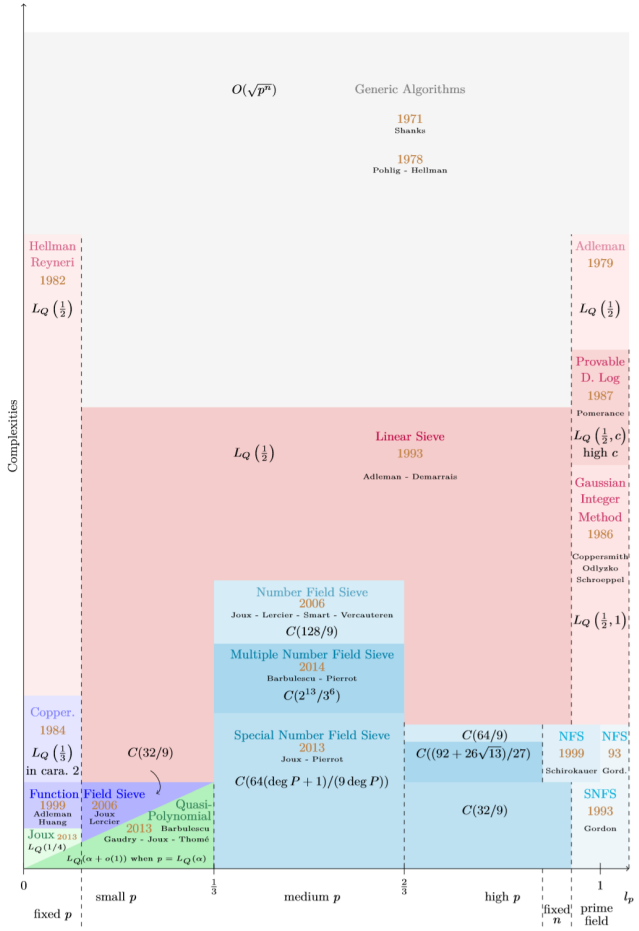
Naive analysis of naive index calculus

- Choose $\log B \approx (\log p)^{1/2}$
- $|\mathcal{F}| \approx B/\log B \approx 2^{(\log p)^{1/2}-(\log\log p)^{-1/2}} \approx 2^{(\log p)^{1/2}}$
- B-smoothness probability
  - $u = \log p/\log B \approx (\log p)^{1/2}$
  - $\rho(u) = (\log p)^{-1/2(\log p)^{1/2}} \approx 2^{-1/2(\log p)^{1/2}(\log\log p)}$
- Number of random trials to get $|\mathcal{F}|$ relations is

$$\approx |\mathcal{F}|\rho(u)^{-1} \approx 2^{(1/2+o(1))(\log p)^{1/2}(\log\log p)}$$

- Each trial has polytime complexity in $\log p$
- Linear algebra cost is $|\mathcal{F}|^{\omega} \approx 2^{\omega(\log p)^{1/2}}$
- Total cost dominated by relation search
- $B \approx L_p(1/2; c)$ leads to slighly better cost $L_p(1/2; c')$

DLP algorithms for finite fields



ECDLP: state-of-the-art

- We have an $L(2/3)$ algorithm to solve ECDLP over fields $\mathbb{F}_{q^n}$ if q and n have the right size
- In most applications we are interested in ECDLP over either prime fields, or $\mathbb{F}_{2^n}$ with extension degree n prime
- Some algorithms have been suggested in those cases, but their complexity is unknown

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Discrete logarithms and related problems

- Let G be a finite cyclic group of prime order q, and let g be a generator
- Discrete logarithm problem: given G, g and h, find x such that $h = g^x$
- Diffie-Hellman problem: given G, g, g^a, g^b, compute g^{ab}
- Decisional Diffie-Hellman problem: let G, g, g^a, g^b, g^c where c is either ab mod q or random (with equal probability), guess correct distribution with probability significantly bigger than $1/2$

- Solving discrete logarithm is a natural approach to solving CDH or DDH, but there could a priori be other methods

DLP vs CDH

- CDH is *roughly* equivalent to DLP
  - Given CDH solver and some precomputation (feasible for the groups used in practice), can solve DLP
- Sketch:
  - Using oracle $CDH(g^a, g^b) = g^{ab}$, can build $CDH(g^x, g^x) = g^{x^2}$, then $CDH(g^{x^2}, g^{x^2}) = g^{x^4}, \ldots$
  - Can build g^{x^e} with "square-and-multiply in exponent" for any e
  - If $|G| - 1$ is smooth, fix g_0 and use Pohlig-Hellman to find x_0 such that $g_0^{g^{x_0}} = g^x$ ie solve DLP in exponent
  - In general: fiind smooth order elliptic curve modulo $|G| - 1$, with x the x-coordinate of a point
- See Maurer *Towards the Equivalence of Breaking the Diffie-Hellman Protocol and Computing Discrete Logarithms*

Remember: El Gamal Encryption

- Public key Alice is (G, g, g^a)
- Private key Alice is a
- (Probabilistic) encryption of $m \in G$ is (c_1, c_2) where
  - $c_1 = g^b$ for some random b
  - $c_2 = mg^{ab}$
- Alice can decrypt (c_1, c_2) with private key
  - Compute $c_1^a = g^{ab}$
  - Compute $c_2/g^{ab} = m$

Static Diffie-Hellman

- Usual Diffie-Hellman oracle: given **random** g^x and g^y, return g^{xy}
- **Static** Diffie-Hellman oracle: given g^x, returns g^{ax} **for some fixed a** (less powerful than DH oracle)
- Can be built using ElGamal decryption oracle
  - Send (g^x, c_2) for arbitrary c_2 to decryption oracle, get $m = c_2/g^{ax}$, and deduce $g^{ax} = c_2/m$

  (remember IND-CCA security definition: adversary may query decryptions of their choice)
- Does this oracle make DLP (g, g^a) easier?

Static Diffie-Hellman

- If $|G| - 1 = uv$, can solve DLP in time $u + \sqrt{v}$ with u calls to the static Diffie-Hellman oracle
  - If $v \approx u^2$ this gives a $O(|G|^{1/3})$ DLP algorithm with $O(|G|^{1/3})$ calls to the oracle
- Idea:
  - Use SDH oracle to produce g^{x^u}
  - Let $x = g_0^{x'}$, with g_0 generator of $\mathbb{Z}^*_{|G|-1}$
  - x^u lies in order v multiplicative subgroup
  - Get x' mod v with "Baby Step Giant Step in exponent"
  - Reduce to a DLP in subgroup of order u, which is solved again with BSGS
- See Brown-Gallant, *The Static Diffie-Hellman Problem*

DLP vs DDH

- There exist groups where DLP is believed to be hard but DDH is easy
- Indeed for elliptic curve groups equipped with an efficient symmetric pairing, we have

$$(P, aP, bP, cP) \text{ is a DDH tuple} \Leftrightarrow e(P, cP) = e(aP, bP)$$

Other "DLP variants'

- The cryptography literature is rich of variants of DLP/CDH problems
- Purpose is provable security: a protocol cannot be proved secure based on DLP only, so a variant is used instead
- Of course:
  - This only makes sense if variant is indeed hard
  - Confidence in hardness only gained by cryptanalysis
- Standard practice: show the variant is hard in the "generic group model"
- See Koblitz-Menezes papers ...

Factorization vs Discrete logarithms

- Discrete logarithm and factorization problems may seem unrelated, yet algorithms to solve them are similar
- Exception (?): quasi-polynomial time algorithm for discrete logarithms in small to medium characteristic
- Hardness of large characteristic field discrete logarithms and integer factorization is comparable today

Pollard's rho

- Factorization problem: given $n = pq$, find p and q
- Idea: find x and y such that $\gcd(x - y, n) = p$
  in other words $x = y \bmod p$ but $x \neq y \bmod n$
- Define some "pseudorandom" iteration function f
- Compute iterates x_i and x_{2i}
- Simultaneously compute $\gcd(x_i - x_{2i}, n)$
- By birthday's paradox,
  $x_i = x_{2i} \bmod p$ after $O(p^{1/2})$ steps on average,
  $x_i = x_{2i} \bmod n$ after $O(n^{1/2}) = O(p)$ steps on average
- Hence we succeed after $O(p^{1/2})$ steps on average

Pollard's $p-1$ method

- Factorization problem: given $n = pq$, find p and q
- A number $x = \prod p_i^{e_i}$ is B-powersmooth if $p_i^{e_i} < B$
- The method assumes that $p-1$ is B-powersmooth, and $q-1$ is not
- Let s be the product of all $p_i^{e_i} < B$
- By assumption $(p-1)|s$, hence $g^s = 1 \bmod p$ (unless $\gcd(g,p) = p$ in which case we deduce p)
- We deduce $p \mid \gcd(g^s - 1, n)$
- First step in gcd computation is $g^s - 1 \bmod n$, computed using square-and-multiply algorithm
- Only works if some factor p such that $p-1$ smooth; elliptic curve method (ECM) removes this constraint

Other factorization methods

- Quadratic field sieve $L_n(1/2)$
- Elliptic curve factorization method $L_p(1/2)$
- (General) number field sieve $L_n(1/3)$

Outline

Introduction

The birthday paradox

Solving discrete logarithm problems

Discrete logarithm variants and factoring

Conclusion

Other cryptanalysis tools

- Above slides only cover a sample of cryptanalysis tools
- Many more cool algorithms to tackle the core computational problems
  - Coppersmith's techniques
  - Groebner basis algorithms
  - Function field sieve
  - Multicollision attacks
  - Gentry-Szydlo's algorithm
  - . . .
- In practice, security often relies on variants easier to solve

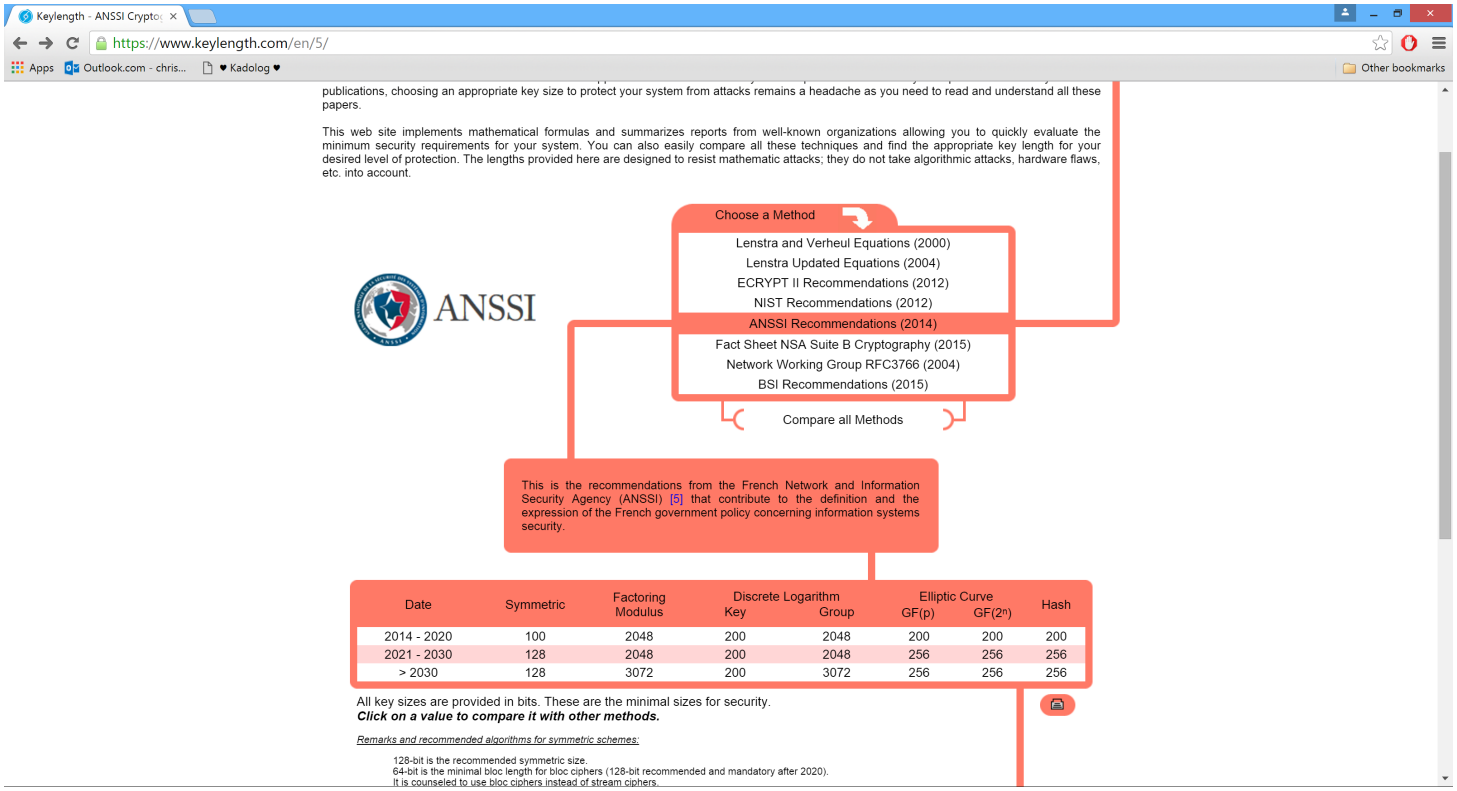
Selecting cryptographic key lengths

- Reasonably easy for symmetric key cryptography, much harder for public key cryptography
- Efficiency vs security tradeof
- Understand complexity of best cryptanalysis algorithms, both asymptotically and in practice
- Tailor key lengths to your needs
  - Short-term or long-term security?
  - Facing the NSA or your little brother?
- Account for Moore's law and algorithmic progress
- Take some security margin
- See Lenstra-Verheul, *Selecting Cryptographic Key Sizes*

Public challenges

- Goal: encourage cryptanalysis attempts to estimate concrete hardness of a problem, and parameters to be used in practice
  - RSA factoring challenges (ended 2007)
  - Certicom ECDLP challenges (solved up to 109 bits)
  - Darmstadt lattice challenges
  - Fukuoka MQ challenges
- Contribute and enter "Hall of Fame" of any of them!

Recommended key lengths



- Check `www.keylength.com` for updates!

Conclusion

- Cryptanalysis attempts are essential for cryptography
- Cryptanalysis relies on a wide range of mathematical tools
- DLP algorithms much more efficient for finite fields than elliptic curves
- Many (large characteristic) DLP algorithms have an analogous factoring algorithm
- Beware of "problem variants" that have been little (if at all) studied